Plan 08 v0: Learned Memory Wrapper for Long-Context RCA

Progress / Fact Record

Mingjia Shi

RCA Foundation Model / Long-Context Memory

July 15, 2026

Combined progress / fact record

Weekly inputs are appended below in chronological order.

Reporting Period: May 2026 Week 4

Date range: Mon 2026-05-25 to Sun 2026-05-31
Month/week is determined by the Monday of the reporting week.

Background

- RCA and debugging tasks are becoming long-context problems: logs, traces, stack traces, test failures, code context, topology, and prior observations often exceed a practical prompt budget.
- Direct long-context prompting is expensive and brittle: high prefill cost, large KV cache, lost-in-the-middle behavior, and no persistent learning between chunks.
- Directly fine-tuning an 8B base on our small RCA/debug data is risky because we do not have the original pretraining mix needed to protect the base model's general capability.
- This project starts from a frozen 7B/8B open-source base and trains a separate wrapper memory module.

Significance

- **Research value**: formulate long-context compression as next-state prediction over learned memory states:

$$m_t = G_\phi(m_{t-1}, c_t), \qquad y_t = F_\theta(q_t, m_t)$$

- **Product value**: build an RCA foundation model for structured root-cause analysis, with lower context cost and better evidence retention.
- **Risk control**: train only the wrapper in v0, keeping the base model frozen to reduce catastrophic forgetting.
- **Community impact**: release a model, benchmark protocol, demo, and technical report rather than only internal experiment logs.

Progress / Fact Record

- **Done**: scoped Plan 08 v0 as a learned-memory wrapper project rather than full self-modifying model training.
- **Done**: read the `rca-demo` SFT results as project initialization: SFT can teach RCA signals, but it is not always a win and is not enough for robust RCA foundation-model capability.
- **Done**: defined two deliverable tracks:
  1. Research paper: new architecture for memory-state next prediction.
  2. RCA foundation model: public RCA datasets first, with public debug traces as a later extension.
- **Done**: added English and Chinese notes plus a dedicated v0 budget.
- **Done**: created the progress/fact-record slide structure.
- **Not verified yet**: no wrapper training result, LoCoMo result, public debug-trace result, or compression demo metric has landed yet.

Status Legend

- **Done**: implemented, documented, or measured from existing artifacts.
- **Verified**: backed by a completed run, metric, or reproducible output.
- **Planned**: design direction or future milestone, not necessarily immediate.
- **TODO**: near-term action to execute next; should be actionable this week / next week.
- This deck separates project facts from plans so readers do not confuse proposals with results.

Status Summary

Done

- Plan 08 v0 scoped.
- RCA-demo SFT boundary read.
- Research/project tracks defined.
- EN/ZH notes and v0 budget written.
- Slide structure created.
- Demo scripts identified.

Planned

- Frozen 8B base + wrapper path.
- LoCoMo / LongBench scaling.
- RCA foundation-model path.
- Trainable wrapper memory.
- Research paper after validation.

TODO

- Select first demo case.
- Generate four prompt variants.
- Run $q25_{base}$ / $q25_{mix}$.
- Score demo outputs.
- Package comparison table.

RCA-Demo Initialization: SFT Boundary Search

- Goal: use `rca-demo` to find how far standard SFT can push a general model toward RCA behavior.
- Setup:
  - Displayed base family: Qwen2.5-7B-Instruct (q25).
  - 4 RCA datasets: Nezha, OpenRCA-500, RCAEval, lincyaw/rca.
  - Training schedules: base, curriculum chain, and joint mix.
  - Evaluation matrix: RCA metrics + general capability benchmarks.
- Project role: this is not the final solution; it is the initialization phase that maps the SFT capability boundary.

SFT Wins: The Model Can Learn RCA Signals

- $q25_{mix}$ became the strongest complete row by `recommendation_f1`:
  - OpenRCA-500: 0.305
  - RCAEval: 0.733
  - lincyaw: 0.242
- After base-mismatch re-eval, $q25_{curr\_4\_lincyaw}$ showed real cumulative curriculum signal:
  - lincyaw `service_set_f1`: 0.272 pre-fix $\rightarrow$ 0.563 post-fix.
  - lincyaw `primary_service_match`: 0.454 $\rightarrow$ 0.723.
  - RCAEval `severity_match`: 0.041 $\rightarrow$ 0.818.
- Takeaway: SFT is useful for injecting RCA structure, service identification, and recommendation behavior.

SFT Boundaries: Not Always a Win

- General benchmark regression appears after direct RCA SFT:
  - $q25_{mix}$ GSM8K drops $0.795 \rightarrow 0.715$.
  - $q25_{mix}$ TruthfulQA drops $0.647 \rightarrow 0.589$.
- Takeaway: SFT can teach RCA behavior, but general-capability regression must be tracked before scaling.

Why Wrapper Memory Follows From RCA-Demo

- Direct base-model SFT exposes three project risks:
  1. capability drift without the original pretraining distribution;
  2. output-format fragility under LoRA updates;
  3. evidence-grounding loss even when JSON surface form improves.
- Frozen base + wrapper separates concerns:
  - base model keeps general reasoning and coding capability;
  - wrapper absorbs long context, tool observations, and RCA/debug traces;
  - failures can be isolated to memory compression instead of the whole model.
- This turns `rca-demo` from a final SFT path into evidence for the RCA foundation-model architecture choice.

First Delivery: A Demo, Not the Full System

- **Planned**: first delivery is a small, convincing demo, not a full paper-quality model release.
- The demo is intended to test the direction:
  1. long context is expensive / brittle;
  2. simple SFT has a capability boundary;
  3. frozen base + wrapper memory is a plausible next step;
  4. the same incident can be compared across full context, summary, retrieval, and memory.
- **TODO**: produce one reproducible case, one comparison table, and one qualitative walkthrough as the next delivery.

Demo Segment 1: Problem Setup

- **TODO**: pick one RCA/debug case with enough evidence to be genuinely long-context.
- Show the raw context shape:
  - issue or incident description;
  - logs / traces / failing test output;
  - code or service context;
  - distractor evidence.
- **Not verified yet**: we have not selected the demo case or measured its token/KV footprint.
- Management question: can the model identify root cause without reading everything directly?

Demo Segment 2: Baseline Comparison

- **TODO**: run the frozen 8B base with three non-learning strategies:
  1. full context;
  2. fixed-budget summary;
  3. retrieval top-k chunks.
- **TODO**: measure:
  - answer quality and evidence grounding;
  - prompt tokens / estimated KV cost;
  - format stability.
- **Not verified yet**: no full-context / summary / retrieval result has been run for the demo case.

Demo Segment 3: Wrapper Memory Path

- **Planned**: process the same context as a chunk stream:

$$m_t = G_\phi(m_{t-1}, c_t)$$

- **Planned**: generate the RCA/debug answer from compressed memory:

$$y_t = F_\theta(q_t, m_t)$$

- **TODO**: use deterministic memory compression for the immediate demo path.
- **Planned**: replace deterministic memory with trained explicit memory tokens after the demo.
- **Not verified yet**: no trained wrapper output has been measured.
- Constraint: keep the base model frozen; all adaptation happens in the wrapper.

Demo Segment 4: What We Need to Show

- The demo does not need to beat every benchmark yet.
- **Target, not verified**: it needs to show a credible direction:
  - lower prompt/KV cost than full context;
  - better evidence retention than naive summary;
  - more complete causal reasoning than retrieval-only;
  - no base-model finetuning required.
- **Decision gate**: only scale to LoCoMo / LongBench / public debug traces after the demo beats cheap baselines on at least one meaningful axis.

Demo Segment 5: Deliverable Package

- **TODO**: demo script that builds one long-context case and produces four prompts.
- **TODO**: output table comparing full context, summary, retrieval, and wrapper memory.
- **TODO**: qualitative page showing where each strategy succeeds or fails.
- **TODO**: short technical note explaining whether the result motivates wrapper training.
- **Decision**: continue to wrapper training only if the demo beats the cheap baselines on at least one meaningful axis.

Demo Assets Already Available in RCA-Demo

- **Done**: `rca-chat` Gradio probe exists and can load matrix checkpoints:
  - $q25_{base}$, $q25_{mix}$, $q25_{curr\_*}$;
  - public RCA datasets plus optional private Liang DTS bundle.
- **Done**: long-context builder script exists:
  - `rca-demo/scripts/compression/build_long_context_rca_benchmark.py`;
  - converts prepared `cases.jsonl` into chunked RCA items with distractors.
- **Done**: prompt-strategy preparation script exists:
  - `rca-demo/scripts/compression/prepare_compression_eval.py`;
  - emits `full_context`, `summary`, `retrieval`, and `learned_memory` prompt rows.
- **Not verified yet**: no end-to-end generation/score run has been completed for this demo path.

Demo Case Selection

- **Preferred public demo**: one RCAEval case.
  - It has microservice fault-injection structure.
  - It supports service/fault-type metrics.
  - It is public and suitable for release.
- **Alternative public demo**: one OpenRCA-500 case.
  - Wider service vocabulary.
  - Good for showing service-level root-cause ranking.
- **Internal-only qualitative demo**: Liang / Nokia DTS.
  - Useful for realism.
  - Not releasable; do not include raw prompts or per-case outputs in public slides.

Demo Build Commands

Step 1: build long-context RCA items

```
python3 rca-demo/scripts/compression/build_long_context_rca_benchmark.py \
  --cases runs/rcaeval_v1/prepared/cases.jsonl \
  --out runs/long_context_rca_demo/items.jsonl \
  --chunk-chars 1800 \
  --overlap-chars 150 \
  --distractors 2
```

Step 2: prepare four prompt strategies

```
python3 rca-demo/scripts/compression/prepare_compression_eval.py \
  --input runs/long_context_rca_demo/items.jsonl \
  --out runs/long_context_rca_demo/prompts.jsonl \
  --summary-chunks 8 \
  --retrieval-k 8 \
  --memory-records 8
```

Demo Four-Way Comparison

Strategy	What it uses	What we learn
Full context	All chunks in prompt	Upper-bound quality, high token/KV cost.
Summary	Fixed-budget compressed text	Cheap baseline; may lose evidence details.
Retrieval	Top-k salient chunks	Good for evidence lookup; may miss causal synthesis.
Learned memory	Rendered wrapper memory state	Whether memory can retain evidence at lower context cost.

Not verified yet: the current `learned_memory` row is a deterministic memory-compressor baseline until the trainable wrapper is run.

Demo Output Layout and Metrics

- **Planned output layout** mirrors matrix cells:
  - `runs/long_context_rca_demo/eval/<model_id>/<strategy>/predictions.jsonl`
  - `metrics.json`
  - `case_scores.jsonl`
  - `eval_command.txt`
- **Metrics to report**:
  - RCA quality: `recommendation_f1`, `root_cause_f1`, `primary_service_match`, `service_set_f1`;
  - grounding: `evidence_overlap`;
  - format: `json_parseable`, `json_repaired`, `degenerate`;
  - compression: prompt token estimate, compression ratio, latency if available.
- **TODO**: wire `prompts.jsonl` into generation and scoring.

Demo Model Choice

- **Baseline model for first demo**: $q25_{mix}$.
  - Existing matrix says it is the strongest complete RCA row by `recommendation_f1`.
  - RCAEval `recommendation_f1` $= 0.733$.
  - JSON format is stable in the public RCA matrix.
- **Control model**: $q25_{base}$.
  - Shows how much RCA behavior came from SFT.

Core Architecture

Memory update

$$m_t = G_\phi(m_{t-1}, c_t)$$

$$y_t = F_\theta(q_t, m_t)$$

- F_θ: frozen 7B/8B base model.
- G_ϕ: trainable wrapper.
- m_t: compact memory state.
- c_t: new context chunk or tool observation.

V0 memory type

Start with explicit memory tokens.

- Easy to train and ablate.
- Compatible with frozen decoder models.
- Safer than writing into weights.
- Hidden-state memory and LoRA memory are later ablations.

Why Freeze the 8B Base?

- We start around 8B because it is strong enough for reasoning and still feasible for repeated experiments.
- Candidate bases:
  - Llama-3.1-8B-Instruct as a secondary validation base.
  - Qwen2.5-Coder-7B or similar for public debug-trace extension.
- The base remains frozen in v0.
- Training only the wrapper limits catastrophic forgetting and makes the contribution easier to isolate.

Two Tracks

Track A: Research Paper

- New memory-wrapper architecture.
- Next-state prediction over memory states.
- Evaluate on long-context memory benchmarks.
- Main claim: compact learned memory can replace part of long prompt context.

Track B: RCA Foundation Model

- Public RCA model with internal qualitative probe support.
- Use code issues, stack traces, failing tests, patches.
- Direct prediction and agentic tool-use modes.
- Release model, report, and demo.

Dataset Plan

Track	Datasets	Purpose
Base long-context	LoCoMo, LongBench, RULER, Needle-in-a-Haystack, Qasper, NarrativeQA	Prove wrapper memory before domain-specific RCA.
Public debug traces	SWE-bench Verified, BugsInPy, Defects4J, QuixBugs, curated CodeNet/APPS traces	Later extension for debugging and trace-based root-cause reasoning.
RCA domain	Nezha, OpenRCA-500, RCAEval, lincyaw/rca	Transfer compression method to RCA evidence bundles.
Internal probe	Liang / Nokia DTS	Qualitative validation only; not for public release.

Evaluation Modes

Direct prediction

Input: issue text, code context, stack trace, logs, failing test output.
Output: root cause, evidence, fix recommendation.

Agentic RCA

The model can call tools such as search, open file, run tests, and inspect traces.
Test-time training updates the wrapper from tool observations; the frozen base does not change.

Baseline comparison

Full context vs. summary vs. retrieval vs. learned memory.

Success Metrics

Quality

- RCA / debug root-cause accuracy.
- Fix recommendation quality.
- Evidence grounding.
- JSON / schema stability for RCA outputs.

Compression

- Token and KV-cache reduction.
- Latency reduction.
- Early / middle / late evidence retention.
- Robustness to chunk order and distractors.

Budget Estimate

Scope	Time	GPUh	Cost	Output
Lean MVP	4–7 wks	1,420–2,950	\$4.3k–\$8.9k	One wrapper, smoke + RCA demo.
Paper-quality v0	7–13 wks	2,820–5,750	\$8.5k–\$17.3k	Long-context + RCA foundation model + ablations.
Strong release	10–16 wks	5,000–9,000	\$15k–\$27k	Multiple seeds, model card, polished report.

Cost model: H100 at \$3 per GPU-hour.

Execution Timeline

Phase	Duration	Decision gate
0. Smoke	3–5 days	Wrapper beats same-length summary on synthetic / NIAH.
1. Long-context	2–3 weeks	Beats summary on LoCoMo / LongBench / RULER.
2. Public debug traces	2–4 weeks	Improves direct root-cause / fix prediction on public debug traces.
3. RCA domain	1–2 weeks	Works on public RCA datasets with at least 4x compression.
4. Release polish	1–2 weeks	Ablations, demo, model card, technical report.

Current Status

- Plan 08 v0 has been scoped as a learned-memory wrapper project.
- English and Chinese notes are maintained in the research notes repo.
- Budget is separated from the full self-modifying-LLM version.
- RCA foundation-model path is defined around public RCA datasets first, with public coding/debug data as a later extension.
- Initial RCA compression scripts and report templates are drafted in the RCA demo repo.
- RCA-demo SFT initialization has identified the boundary: useful RCA learning exists, but direct SFT is not robust enough as the final RCA foundation-model route.

Next Steps

1. Build the first demo case and four prompt strategies.
2. Run frozen 8B base baselines: full context, summary, retrieval.
3. Add wrapper-memory path for the same case.
4. Package the demo table and qualitative walkthrough.
5. After demo validation, scale to LoCoMo / LongBench and public debug-trace datasets.

Ask / Discussion

- Is frozen-base + wrapper the right first milestone?
- Which 8B base should be the first public model target?
- Should the first paper emphasize long-context memory, RCA foundation modeling, or both?
- Which demo case would be most convincing for external readers?

Reporting Period: June 2026 Week 1

Date range: Mon 2026-06-01 to Sun 2026-06-07

Weekly meeting logic: why this matters → what we are doing → goal → progress.

Updates compiled on 2026-06-04/05 belong to this same Week 1 report.

Recall: The Two Core Problems

Problem 1: long-context inference

At test time, the model may need to answer from very long evidence: logs, traces, documents, retrieved cases, or code. The question is how to use that evidence without always paying full-context cost.

Problem 2: catastrophic forgetting during training

When we train or adapt a model for a specialized setting, it may improve on that setting but lose general abilities such as reasoning, retrieval, or instruction following.

- Root-cause-analysis (RCA) benchmarks matter here because they naturally combine both problems: long evidence at inference time and specialized adaptation pressure at training time.

Preliminaries: Vocabulary For This Talk

Term	Meaning in this project
Frozen base model	The original LLM is not updated; this protects its general ability.
Adaptation module	A small extra component trained for the task while the base stays frozen.
Learned memory module / wrapper	The adaptation module tested here; it compresses long evidence into soft memory tokens.
Soft memory / soft prompt	Continuous vectors inserted into the model, not human-readable text.
Gate	A decision rule that chooses whether to use the learned memory or fall back to the base/full-context path.
Do-no-harm	The gate should keep gains when memory helps and avoid losing points when memory is unsafe.

Background: Why Study These Two Problems?

Problem

RCA-style tasks are a motivating benchmark family because they expose both long-context inference and training-time forgetting. They require long evidence at test time, and task-specific training can still damage general model behavior.

- We want task adaptation without overwriting the frozen base model.
- The useful module should compress long context, be reusable, and be safe to turn off when it is not appropriate.
- This connects May Week 4 to June Week 1: first build a wrapper, then measure when it is safe to use.

What Is The Use?

Practical use

A learned memory module could make long-context inference cheaper and more modular: keep the base model frozen, attach a small learned memory path, and reuse compressed evidence.

- Avoid full-context cost when gist-level memory is enough.
- Preserve a fallback path to the original base model.

Research use

It gives a clean testbed for frozen-base adaptation: what can a fixed-size soft memory encode, and when does it fail?

- Positive result: wrapper can learn.
- Boundary result: fixed memory has a capacity wall.

What Are We Doing?

1. **Train a learned memory module** around a frozen base model: long evidence context $c_{1:n}$ is compressed into soft memory m.
2. **Evaluate the wrapper** against no-context, full-context, retrieval, and Gist-style baselines.
3. **Characterize the boundary**: where memory helps, where it is neutral, and where it collapses.
4. **Add a gate**: only let memory participate when intrinsic signals suggest compression is safe.

Short version

We are not claiming "memory replaces context." We are building controlled frozen-base adaptation: learn memory, measure its limits, then gate it.

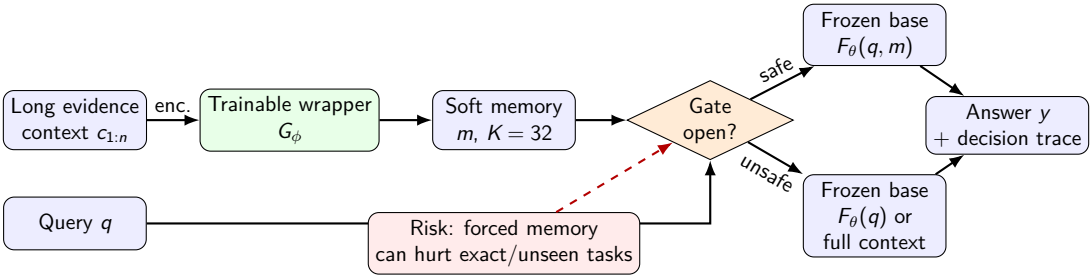
Goal For This Line Of Work

Main goal

A frozen-base memory system that improves long-context cases where compression is useful, while preserving the base model when memory is unsafe.

- **v1**: characterize the learned soft-prompt wrapper; write the result as a bit-capacity-wall paper.
- **v1.5**: build a do-no-harm gate between wrapper and base.
- **v2**: extend from one-shot compressed memory to cross-session latent memory with read/write behavior.

Architecture: Input, Memory, Gate, Output



How Is It Going? v1 Result

Benchmark	Wrapper	Gist	Best reference
QuALITY	0.193 ± 0.032	0.180 ± 0.044	no-context 0.141
MuSR-mm	0.493 ± 0.008	0.501 ± 0.013	full-context 0.551
RULER-NIAH	0.000 ± 0.000	0.000 ± 0.000	full-context 0.995

- **Good news**: the wrapper can train and helps on compressible gist-style inputs.
- **Lesson**: it is not a universal replacement for long context; exact-retrieval cases expose a capacity wall.
- **Setting**: P08-S2; full facts in v1 results summary.

How Is It Going? Interpretation

1. **Wrapper can learn**: memory is useful when the answer can be represented as compressed/gist information.
2. **Wrapper changes inference**: it is not neutral; the learned memory path can pull the base away from safer behavior.
3. **Always-on memory is risky**: if the task needs exact details or unseen reasoning transfer, forced memory can lose information.
4. **Next design**: put a gate between wrapper and base, and open it only when intrinsic signals say compression is safe.

How Is It Going? v1.5 Gate

Signal family	Interpretation	Direction	Use
Confidence / seq logprob	wrapper likely correct	positive	open
Wrapper-to-base KL / JS / TV	wrapper fights base	inverse	close
Memory/write dynamics	wrapper competence signal	positive	open/close
Mid/late-layer drift	late reasoning distorted	inverse	close

- **Goal**: keep in-distribution gains while avoiding OOD harm.
- **Setting**: P08-S3/P08-S6 for signal probes; detailed v1.5 results continue in the appended gate section.
- Main deliverables: signal grids and v1.5 PDF.

This Week's Status Summary

Done

- v1 reframed as bit-capacity characterization.
- Main result cells, settings, and terminology are linked.
- Gate direction is now concrete, not vague.

Next

- Polish v1 paper story.
- Run/validate do-no-harm gate.
- Test if multi-task training widens the safe region.

Meeting takeaway

The project is moving from "can memory learn?" to "can memory be used safely?" The current answer is positive but bounded: useful compression exists, and the gate is the mechanism that can make it deployable.

Where To Read More

Item	Link
Plan 08 hub	README / deliverables hub
v1 facts	main results summary (setting P08-S2)
v1.5 report	compiled PDF (setting P08-S3)
v1.5 signal grid	CSV + figures (setting P08-S3)
v2 setting	v2 plan (setting P08-S4)

Detailed Question: Can Learned Memory Be Deployed *Safely*?

- A frozen base + a small learned **memory wrapper**.
- **The question:** can we get its gains *where it works* and *never lose points* where it doesn't?

One-line answer: **mostly** — an intrinsic-signal gate keeps the wrapper's *in-distribution gains* and *recovers most of the out-of-distribution harm* (oracle shows full headroom), and the gate signal **transfers across 7 model families with no tuning**.

→ every term/number on the next slides links out to a wiki page: `summary/2026-06-05/v1.5-glossary.md`.

Preliminaries For The Gate Evidence

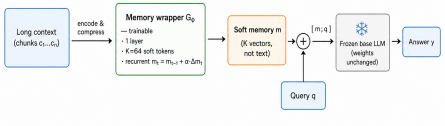
Term	Meaning in the detailed gate slides
In-distribution (ID)	Test inputs similar to what the memory module was trained on.
Out-of-distribution (OOD)	Test inputs that differ from training; this is where memory can become unsafe.
Intrinsic signal	A number read from the model's own forward pass, without needing the true answer.
Memory-write dynamics	How strongly the module changes or writes memory; used as a clue for competence.
Model-agnostic	The same gate rule should transfer across model families without retuning.
AUROC	A threshold-free score for whether a signal separates useful vs unsafe memory cases.
Leave-one-model-out	Train/choose the signal on some model families, then test on a held-out family.

Why it matters

- **The broader risk:** adapting a model to a task (e.g. SFT) can *overwrite* general ability — reasoning, retrieval, instruction-following (catastrophic forgetting). A **frozen base** sidesteps this: adaptation lives only in a removable memory module.
- **Frozen base = a promise: don't regress.** A memory/adaptation module is only deployable if it can't quietly *hurt* the model on inputs it wasn't built for.
- **Our v1 finding:** the learned memory module is a *narrow lossy compressor* — it helps on some inputs and **actively hurts** on others (e.g. exact-answer QA). Negative transfer is the blocker.
- **So the real product is not just "a better memory module"** — it's **knowing when to use it**, automatically, on any base model.

→ v1 characterization (3-regime law): analysis `v1-results-2026-06-03.md`; per-bench data `mem-test/mem-embedding/summary/v1-archive/tables/*.csv`. *(Training-time forgetting itself is measured in the separate Janus track, not this paper.)*

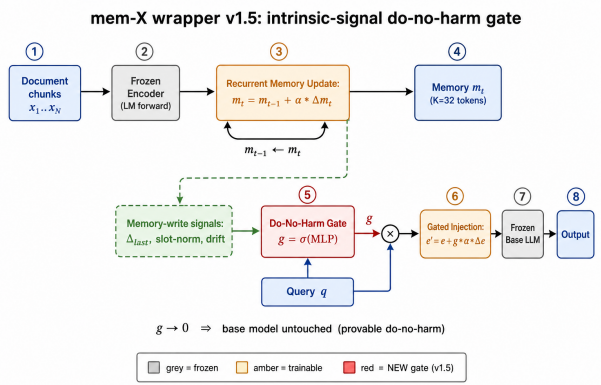
How the memory wrapper connects to the base



- G_ϕ is a **small (1-layer, K=64) recurrent module** that compresses the chunks into *K soft memory vectors* (not text).
- Those vectors are **prepended to the query** as a soft prompt ($[m; q]$); the **base is frozen** — only G_ϕ trains, so adaptation can't overwrite general ability.

→ TikZ source `summary/2026-06-05/wrapper-connect.tex`; combine modes `mem-test/mem-embedding/src/mem_embedding/wrapper.py`; canonical recipe `P08-S1` (Qwen3-8B, K=64, `combine=xattn`).

What we do (architecture)



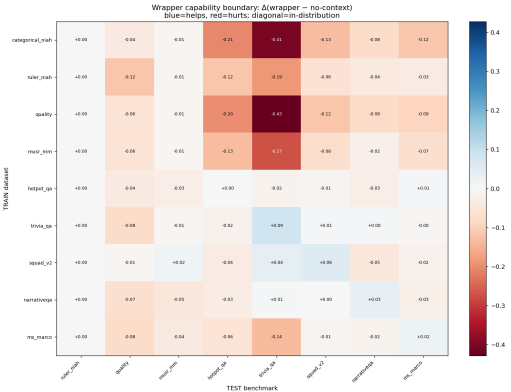
Recurrent memory (carried from v0) + a **do-no-harm gate** g: $e' = e + g\alpha\Delta e$, with $g \to 0 \Rightarrow$ the base is *untouched*. → diagram source `summary/2026-06-05/architecture-v1.5.tex`; v0→v1.5 diff in `summary/2026-06-05/v1.5-glossary.md`.

The bet — why this should work

- **Hypothesis:** whether the wrapper will help is *readable from the model's own forward pass* — specifically from how much it **wrote into memory** ($\|\Delta m\|$, slot norms, drift).
- If true, a tiny gate on those signals can decide **per input** whether to open the wrapper or fall back to the base.
- **Crucial requirement:** those signals must be *the same across model families* — otherwise we'd re-tune per model and it's not deployable.

→ data `raw/grids-xmodel-2026-06-05/xmodel_consistency_7models.csv`; analysis `summary/2026-06-04/v1.5-metric-candidates-2026-06-04.md` (codebook+verdicts) · matrix §2/§2b · setting P08-S6.

Result 1 — *where* it helps (expected vs. real)



Expected: helps where trained, decays as test drifts from train.

Real (72 train×test cells):

- **in-distribution: it helps** (QA +2 to +9 pt).
- competence is **distribution-bound**: gain → harm already at the same-task line.

→ data `raw/grids-xmodel-2026-06-05/transfer_matri` (+ `raw/grids-xmodel-2026-06-05/transfer_lo` analysis matrix §8 · setting P08-S7.

Baselines side-by-side: floor → wrapper → gate → ceiling

Qwen3-8B; wrapper trained on *mix* (all datasets). Numbers = native score (PRIMARY metric).

bench	reg	no-ctx (floor)	ungated wrapper	gated (route)	oracle (gate ceil.)	**full-ctx (ceiling)**
quality	A	.233	.227	.227	.293	.200
musr_mm	A	.520	.510	.510	.600	.495
ruler_niah	C	.000	.000	.000	.000	**.995**
hotpot_qa	B	.234	.202	.212	.320	**.623**
trivia_qa	B	.474	.324	.440	.538	**.635**
squad_v2	B	.171	.157	.157	.220	**.656**
narrativeqa	B	.149	.119	.158	.182	.150
ms_marco	B	.220	.201	.222	.261	.282

- **Ceiling (full context)** is huge on exact/extractive QA + the needle (ruler 0→.995, squad/hotpot/trivia ∼.62−.66) but ≈ no help on multiple-choice gist (even hurts quality).
- **Mix wrapper (ungated)** mostly *hurts*; the **gate** pulls it back toward the floor; **oracle** shows headroom. Compressed memory ≪ raw context where exact detail matters — the capacity wall.
- *Ordinary SFT (LoRA on the base) row: running* — the adapt-by-fine-tuning comparison.

→ data `raw/fullctx-2026-06-05/`, `raw/baselines-2026-06-05/gate_baselines.csv`; how to read every number `raw/README.md`.

So what + honest scope + next

Takeaway: a frozen-base memory module becomes **safely deployable** — it captures gains where it's competent and is provably harmless elsewhere, using one *model-general* intrinsic signal.

- **Honest scope:** in-dist gains are *modest & dataset-dependent* (QA yes, multiple-choice no); the gate *recovers most* OOD harm but doesn't fully guarantee do-no-harm under honest CV (17/35; oracle 35/35). The headline value is **safe, modular memory**, not SOTA.
- **Landed:** locking experiment (§7b) — one gate keeps in-dist gain & cuts OOD harm. **Running:** significance (seeds+CIs), K & combine-mode ablations, full-context ceiling.

→ full ledger & asset index: `mem-test/mem-embedding/summary/matrix.md`; decode any term: `summary/2026-06-05/v1.5-glossary.md`.

Reporting Period: June 2026 Week 2

Date range: Mon 2026-06-08 to Sun 2026-06-14

Goal: concise progress record; detailed facts live in linked notes (results-v1.7.3).

This week = rigorous re-validation of the gated compressor + a reframe of the contribution.

Continuity From June Week 1

Week 1 result

The wrapper learns useful compression only in a bounded regime; therefore place a do-no-harm *gate* between wrapper memory and the frozen base.

- Week 2 stress-tested the gated compressor on **agentic tool-use + RCA**.
- We **hardened the evaluation** (leak-free, held-out, matched-budget) and re-ran clean (v1.7.3).
- The clean results moved the story from "build a better compressor" to "**a compressor-agnostic robustness layer**."

This Week: Main Message

Reframed contribution

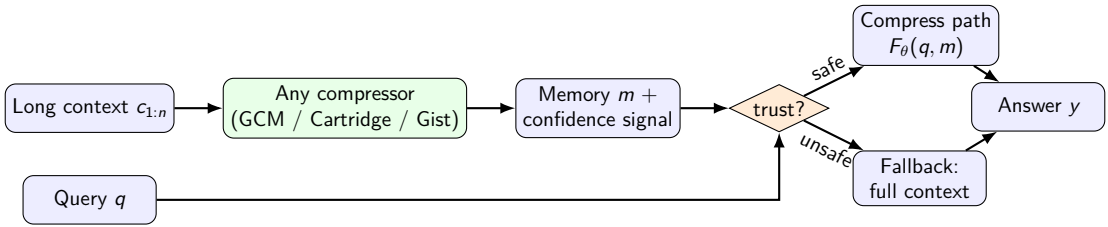
Not a better compressor — a **robustness layer**: an external confidence / self-verification signal that detects when compression is unsafe and falls back to full context (do-no-harm). It works on top of *any* compressor.

- Compression alone is **capacity-bound** and **never beats full** — which is exactly why a detect-and-fallback layer is needed.
- The gate is **compressor-agnostic**: it recovers $\approx$ full on our method *and* on a faithful Cartridge baseline.
- Evaluation hardened to leak-free / held-out / matched-budget; honest numbers throughout.

Why The Reframe: What The Clean Data Shows

- Fixed-size soft-prompt compression is **capacity-bound**: correlation(context length, value-add) $= -0.4$ — it helps at low compression ratio (short context, $\leq 3\times$) and **fails at high ratio** ($\geq 9\times$, long context).
- On every tool benchmark, compressed accuracy stays **below full context**; the realized compute saving from compression alone is ≈ 0.
- So the deployable value is not "compress better" but **"detect bad compression and fall back"** — a safety layer usable with any compressor.

High-Level: Robustness Layer (compressor-agnostic)



Core idea

Any compressor makes memory; a confidence / self-verification signal scores it; the gate compresses when safe and falls back to full when not — never worse than full.

Core Clean Findings (v1.7.3, leak-free, in-task)

Tool benchmark	no-ctx	full	GCM compress	gate gAcc	detect AUROC
BFCL live-multiple	0.01	0.91	0.72	0.89	0.75
BFCL live-simple	0.01	0.91	0.53	0.92	0.88
BFCL simple	0.02	0.99	0.17	0.99	0.52
ToolACE	0.01	0.95	0.14	0.94	0.53

- Compression stays **below full** everywhere; the **gate gAcc** $\approx$ **full** (held-out, within $-0.05..+0.03$).
- Detection works where compression is learnable (live benches AUROC 0.75–0.88); elsewhere the gate **safely abstains** (falls back to full).

Headline: The Gate Is Compressor-Agnostic

Compressor	benchmark	compress	detect AUROC	gate gAcc
GCM (ours)	BFCL live-simple	0.53	0.88	0.92
Cartridge	BFCL parallel	0.28	**0.81**	0.93
Cartridge	BFCL live-multiple	0.51	**0.73**	0.90
Cartridge	BFCL live-simple	0.31	0.70	0.90

Why it matters

The same confidence gate, applied to a *different* compressor (Cartridge), still detects its failures (AUROC 0.6–0.8) and recovers $\approx$ full. The robustness layer is not tied to our encoder — it is a reusable safety net for any compressor.

Evaluation Rigor Added This Week

- **Leak-free splits**: a seed-independent train/eval partition (caught and fixed a train/eval overlap; all tool results re-run clean as v1.7.3).
- **Held-out gate**: 5-fold cross-validated threshold (removes in-sample optimism).
- **Matched-budget baselines**: Cartridge / Gist re-trained at equal data, steps, and memory.
- **Args-aware tool scoring** (function name + arguments) and **multi-seed** confidence intervals (in progress).
- **Breadth**: tool benchmarks from **5 independent teams** (BFCL, API-Bank, ToolACE, Hermes, Glaive) plus RCA.

Next Steps

1. **K-sweep**: map the capacity / compression-ratio frontier — does more memory rescue long context?
2. **Transfer matrix**: in-task / cross-task / cross-domain for the gated system + baselines.
3. **Args-aware verdict**: do the tool wins survive when arguments must also be correct?
4. **Agnostic claim**: apply the gate to a strong external compressor (LLMLingua).
5. **Paper identity**: lock "diagnosis + do-no-harm robustness layer" (well supported by the clean data).

Where To Read More (v1.7.3)

Item	Link
Clean results	results-v1.7.3
Reframed thesis + plan	robustness-plan
Reviewer-response plan	reviewer-response
Setup / recipe / protocol	experimental-setup

Reporting Period

Plan 08 / Paper B — June Week 3

Mon 2026-06-15 to Sun 2026-06-21

Weekly progress / fact record. Detailed facts + settings in the linked GitHub notes.

Recall: Why We Are Doing This

The problem

Feeding an LLM a long context every turn is slow and expensive. We study whether we can **compress the context into a few latent "memory" tokens** a *frozen* base model reads — and, when that compression would lose what the query needs, **fall back to the full context** ("do-no-harm").

- **Last week:** a clean (leak-free, stable-training) study reframed the work from "build a better compressor" to a **capacity-bound diagnosis** + the **do-no-harm gate** it prescribes.
- **This week:** the supporting experiments actually *landed* — cross-model breadth, a module ablation, a *deployable* gate threshold, and an honest read of what is still missing.

Preliminaries: Terms Used Today

- **Frozen base**: the pretrained LLM; weights never change (so behavior is recoverable).
- **Learned memory / soft prompt**: K trained latent vectors ($\approx$ "compressed context") the base reads.
- **Compressor**: the module that maps long context $\rightarrow$ memory (ours = GCM; rivals = ICAE/Beacon/500x/...).
- **Gate**: a per-input decision — *keep* the compressed memory (cheap) or *fall back* to full context (safe).
- **Do-no-harm**: the gate never lowers quality below full context (by construction the base is recoverable).
- **AUROC**: how well a gate *signal* ranks safe-vs-unsafe inputs (0.5 = chance, 1.0 = perfect).
- **recall@P95 / coverage**: how much we can safely compress at $\geq$95% precision (the savings we can claim).
- **ctx>K**: items where the context is longer than the memory budget — the only items where "compression" is real.
- **test-agnostic threshold**: a gate cutoff set *without* test labels (so it is deployable on any dataset).

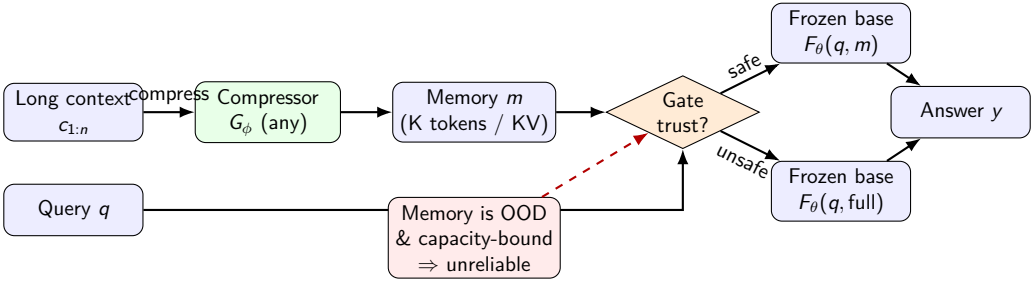
This Week: Main Message

Diagnosis holds across models; the gate is deployable; the net-win is the open gap

The capacity-bound diagnosis **reproduces across 5 model families**; a module ablation shows **only distillation + reconstruction are load-bearing**; and a **label-free gate threshold** gives do-no-harm on every model — but its *savings* are still the piece we must prove.

- **Verified**: no learned compressor (soft-prompt or KV-injection) beats full at real compression (ctx>K); among same-complexity soft-prompt methods our GCM is clearly best.
- **Verified**: cross-model (GLM, Qwen3, Mistral, 2×Llama) + module ablation + a test-agnostic gate threshold.
- **Open (next)**: the cost-quality net-win (tokens saved at matched quality) vs gate baselines (TARG / entropy / LLM-judge).

Architecture: Compressor, Gate, Do-No-Harm Fallback



Idea

The gate reads a cheap failure signal off *any* compressor's memory; opens it only when safe, else falls back to full (never hurts).

Verified: No Compressor Beats Full (by injection TYPE, ctx>K)

(A) Soft-prompt ≈ our complexity

method	bfcl_lm	hermes†
full (ceiling)	0.91	0.93
GCM (ours)	**0.67**	**0.52**
ICAE	0.22	0.67
ComprExIT	0.29	0.33
AOC / Gist / LCC	0.05 / 0.00 / 0.00	0.34 / 0.34 / 0.3

(B) KV-injection: compute ≫ ours

method	bfcl_lm	hermes†
full	0.91	0.93
Beacon	0.31	0.85
Cartridge	0.34	0.46
500x	0.09	0.51

- **No method (soft-prompt or KV) beats full** at real compression; among same-complexity soft-prompt methods **GCM is clearly best**.
- KV-injection (Beacon strongest) costs far more per token, still $<$ full $\Rightarrow$ reported *separately*, not ranked against (A).
- Floors (reference, not methods): no-ctx ≈0.01; trunc (first-K) 0.46. †hermes front-loaded $\Rightarrow$ trunc≈full.

Verified: The Diagnosis Reproduces Across Model Families

model (bfcl_live_multiple)	no-ctx	full	GCM	gate AUROC
GLM-4-9B	0.01	0.85	0.66	0.70
Qwen3-4B-2507	0.00	0.88	0.71	0.82
Ministral-8B	0.02	0.88	0.63	0.79
Llama-xLAM-8B	0.00	0.90	0.76	0.81
ToolACE-8B	0.00	0.88	0.72	0.90

- **5 dense families** (GLM, Qwen3, Mistral, 2×Llama): same story — necessity holds, compress $\ll$ full, do-no-harm gate recovers $\approx$full.
- Real gate detection (high AUROC) concentrates on high-headroom tool benches; the 2 MoE were dropped (95 GB OOM).

Verified: Module Ablation — Only Distillation + Reconstruction Matter

From the full method, remove one module

(Qwen3-8B; Δcompress-acc vs full config; 2 benches)

- − **distillation**: −0.51 / −0.22 **(collapse)**
- − **reconstruction**: gate AUROC 0.75 → 0.48 **(gate dies)**
- − task loss: bench-dependent (keep it)
- depth 16→4, K64→4, random init, −dev: ≈ neutral

Takeaway

Minimal sufficient method = a *tiny* encoder ($N \approx 4$, $K \approx 16$) + distillation + reconstruction (which powers the gate) + task loss. Everything else (big encoder, large K, alignment losses, OOD fixes) is droppable ⇒ supports "minimize-scale; the gate is the contribution."

Verified: A Deployable (Test-Agnostic) Gate Threshold

The deployment question

In production you have no labels — so you cannot tune the gate cutoff τ on the test set. *Is there a τ rule that works on any dataset?*

- **Yes, with a calibrated-probability signal** (base "margin" = top1−top2 prob): a fixed threshold is dataset-independent.
- **margin** $\geq 0.95 \Rightarrow$ **do-no-harm on all 20 model×bench cells** (worst -0.01); coverage *auto-adapts* (compress where safe, fall back where not).
- Intrinsic signals (reconstruction-based) are *not* test-agnostic (arbitrary scale $\Rightarrow$ need per-dataset calibration).
- Signal ranking (T10): agnostic `neg_entropy`/`conf` $\approx$ best intrinsic ($\sim$0.6 mean AUROC); others at chance.

Honest Status: Good News and the Open Gap

Good news (Verified)

- Diagnosis is robust (whole field, 5 families, leak-free + stable).
- Do-no-harm gate works and is *deployable* (label-free τ).
- Method is minimal (distill + recon + tiny encoder).
- Baselines re-implemented faithfully + at one recipe.

Bad news / risk (TODO)

- **Net-win not yet shown**: do-no-harm holds, but at safe τ coverage is low $\Rightarrow$ savings are bounded by signal quality. **This is the make-or-break next experiment.**
- Cross-domain gate transfer (T8c) needs a clean re-run.
- Net-win Pareto vs TARG / entropy / LLM-judge gates = next.

Deliverables and Next Steps

Item	Link
v1.7.5 results	results-v1.7.5.md (T1–T11 + roadmap)
Deploy / teardown	mem-test/deploy/ (PVC bootstrap + recovery, in code repo)

1. **TODO (top):** cost-quality **net-win Pareto** + recall@P95 vs gate baselines (TARG / entropy / LLM-judge) — the payoff claim.
2. **TODO:** clean cross-task/domain gate-transfer matrix; bootstrap CIs on the headline numbers.
3. **Planned:** fold the remaining uniform-recipe benches; finalize the diagnosis-led paper draft.

Reporting Period

Plan 08 / Paper B — June Week 4

Mon 2026-06-22 to Sun 2026-06-28

Weekly progress / fact record. Detailed facts + settings in the linked GitHub notes.

Recap: Last Week → This Week

Where we were

The capacity-bound diagnosis + **do-no-harm gate** reproduced across model families; the open gap was the *net-win* (savings at matched quality) and a real long-context product story.

- **This week (research):** v1.8.0 **main table with full gate metrics** (F1/precision/recall/fallback), per-base HP tuning, and a **flip-one ablation** showing the method is robust — the gate carries the win.
- **This week (product):** shipped a **self-contained, one-command CMG-RCA foundation-model demo** and framed the **hard motivation** (the H100 8k memory wall).
- **This week (infra):** deployed a shared vLLM service for a colleague on an isolated pod.

This Week: Main Message

Under the H100 memory wall, compress + gate is cheaper *and* not worse

On a single 80 GB H100 an 8–9B model fits only ∼8k tokens of full attention; real RCA evidence is far longer. GCM folds the *full* evidence into 256 memory tokens, and the do-no-harm gate keeps quality $\geq$ full — so the contribution (the gate) holds while the system becomes hardware-feasible.

- **Gate metrics (new):** gated-acc $\geq$ full on every bench; on bfcl tool-use the gate even *beats* full (0.86–0.92).
- **Ablation:** flipping any single knob moves compress by only $\pm 0.03 \Rightarrow$ the method is robust; the gate is what matters.
- **Product:** CMG-RCA demo + weights + inference code shipped to GitLab; deployable in one command.

The Hard Motivation: the H100 8k Memory Wall

Measured wall (not a preference)

A 16k full-context pass OOMs on 80 GB; the linear-attention variant reaches $\sim$64 GB at just 2.5k tokens ($O(L^2)$ attention memory). So the *usable* full-attention window for this model class is $\sim$**8k**.

- Internal **CMG-RCA** evidence: median **7.5k**, **max 28,610** tokens; **37% (14/38) exceed the 8k window**.
- Under the wall you must *truncate* (drop most evidence) — unless you *compress*.
- GCM: full evidence $\rightarrow$ fixed **256-token** memory; KV/compute bounded by K, not L $\Rightarrow$ any length fits the H100 budget.

Verified: Compress Beats the Truncated Read Under the Wall (same H100)

condition (internal CMG, n=38, one fixed H100)	module-ID acc	attention tokens
no context (prior)	0.079	$\sim$0
truncated full read (8k, frozen base)	0.105	$L = 8$k
compressed M (full evidence $\rightarrow$ 256)	**0.263**	$K = 256$

- **2.5$\times$ higher accuracy at $\sim$32$\times$ fewer attention tokens** — evaluated on *one* GPU to remove fp-rounding noise.
- Best of all checkpoints on this GPU is the served model (`mt_all_distill1`); the do-no-harm gate then guarantees $\geq$ full.
- Honest: CMG is near-random for the base (frozen-base full $\approx$0.13; $\approx$0.21 even memorizing) $\Rightarrow$ the robust win is the *relation* compress>full>no-ctx.

Verified: Main Table with Full Gate Metrics (do-no-harm holds)

bench (Qwen3.5-9B)	no-ctx	full(trunc)	compress	**gated**	gate F1
bfcl_live_multiple	0.02	0.83	0.71	**0.86**↑	0.87
hotpot_qa	0.29	0.58	0.34	0.66	0.36
squad_v2	0.22	0.64	0.24	0.66	0.21
toolace	0.02	0.91	0.16	0.91	0.00
narrativeqa	0.11	0.14	0.16	0.14	0.00

- **gated $\geq$ full on every bench**; on the headroom bench the gate is discriminative (F1 0.87, prec 0.95, recall 0.81) and *beats* full.
- Low-headroom benches: gate **never fires** (fallback) $\Rightarrow$ do-no-harm by construction. (Qwen3-8B mirrors this.)
- **Flip-one ablation (both bases):** norm/recon/dev/depth/recurrent/K/LoRA/distill each move compress by only ± 0.03 $\Rightarrow$ robust; the gate is the contribution.

Done: CMG-RCA Foundation-Model Demo Shipped

Self-contained, one-command deploy (GitLab `cmg-rca-foundation`)

Everything but the public base model is in the repo: GCM inference code, the trained adapter, the demo server + UI, the 38 CMG cases, and `deploy.sh` (downloads `Qwen/Qwen3-8B`, launches in one step).

- Live interactive demo: per-case *no-ctx* / *truncated* / *compressed-M* sources, streaming RCA reports, chat over M, context-window swimlane, and a metrics panel.
- Zero-downtime ops: hot-swap adapter (`/api/reload`) + hot-update metrics, no restart.
- Honest framing baked in: CMG is a long-context *demo*; the absolute number is ceiling-bound (resource-limited).

Honest Status, Infra, and Next Steps

Verified / done

- Gate metrics + main table; flip-one ablation (robust).
- Memory-wall motivation + same-H100 compress>full.
- Foundation demo + weights + code shipped (one-command).
- Infra: shared vLLM (Qwen3.5-9B, OpenAI API) on an *isolated* pod for a colleague.

Open / running (next)

- **Ceiling is compute+data**, not the method $\Rightarrow$ large headroom from more RCA data + variable-length training.
- Running: adaptive-K (Qwen3.5), QA-bench HP tuning, encode-length robustness.
- TODO: baseline columns (SFT-LoRA / Cartridge / Gist / TARG / Transformer-XL); cross-model gate table.

Reporting Period

Plan 08 / Paper B — June Week 5

Mon 2026-06-29 to Sun 2026-07-05

Leader update: research outcome, long-context facts, v2.0.0 options, and resource ask.

Scientific Question

Main question

When a model faces long evidence, what should it keep, what should it ignore, and how can it know whether its reduced context is still reliable?

Why this is scientific, not only engineering

"Long context" is not one capability. It decomposes into retrieval, local extraction, distributed semantic reasoning, distractor robustness, and architecture-dependent memory access.

- The core object of study is the **interaction between task type, context length, and reduction method**.
- The scientific goal is to map this interaction, then design a method that adapts instead of assuming one fixed rule.

Background: What Did We Test?

Benchmarks = task regimes

- **RULER / NIAH:** synthetic needle retrieval; controls length.
- **numerical / categorical / coding NIAH:** exact value, label, or code-style retrieval.
- **SQuAD v2:** local extractive QA; tests whether a recent window is enough.
- **HotpotQA:** multi-hop QA; evidence is distributed.
- **NarrativeQA / QuALITY:** long, abstractive/global reading; tests focus under distractors.
- **TriviaQA / LoCoMo / MuSR:** noisy evidence, dialogue memory, and reasoning variants.

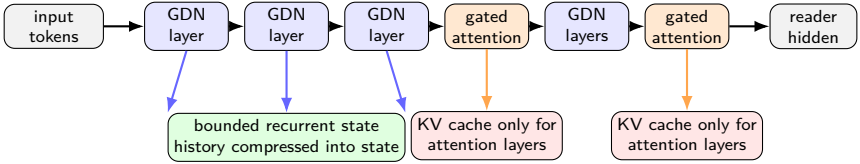
Baselines = context-reduction strategies

- **full / no-context:** ceiling and floor.
- **window / truncation:** assumes answer is local or recent.
- **RAG / BM25:** assumes query and evidence share lexical overlap.
- **Prompt compression (LLMLingua):** assumes token salience predicts usefulness.
- **KV eviction:** SnapKV/H2O/Expected/TOVA, Streaming, Knorm, KVzip/FastKVzip.
- **Token merging:** tests whether similar tokens/spans can be merged without losing the answer.

Readout

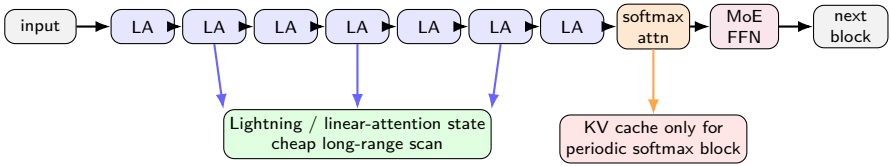
The experiment matrix asks which method fails in which task regime, not only which method has the best average score.

Background: Qwen3.5 / 3.6 Hybrid GDN Stack



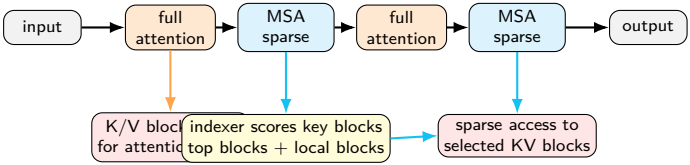
Prefill dependency: most layers scan tokens into a GDN recurrent state; periodic attention layers create ordinary K/V for exact lookup. **KV-cache methods apply only to the attention subset, not to GDN layers.** Qwen3.6 is reported as KV on 16/64 layers, so the KV lever is partial.

Background: MiniMax-Text-01 / M1 Hybrid Lightning Stack



Prefill dependency: the published MiniMax-Text-01/M1 pattern is roughly **7 Lightning/linear-attention blocks + 1 softmax-attention block**, with MoE capacity. Lightning blocks carry cheap long-range state; softmax blocks periodically restore exact token interactions. **KV eviction applies only to the softmax blocks.**

Background: MiniMax-M3 Sparse-Attention Stack



Prefill dependency: M3 is not recurrent/linear. It is a sparse-attention Transformer: sparse layers use an indexer to choose key blocks, then exact attention runs on those selected blocks. **KV exists, but access is already learned and sparse**; KV eviction methods are possible only as a comparison to MSA, not as the native mechanism.

Experimental Facts We Established

- **Fact 1: fixed baselines are regime-specific.** Window, RAG, prompt compression, and KV eviction each work in some regimes and fail in others.
- **Fact 2: length changes method ranking.** Attention-based KV eviction works around shorter contexts but can collapse at 16k+.
- **Fact 3: task type changes method ranking.** Retrieval, extractive QA, global reasoning, and literary MC prefer different context reduction behavior.
- **Fact 4: more context is not always better.** Full context can hurt when the context introduces distractors or exceeds the model's usable reasoning ability.
- **Fact 5: model architecture matters.** Linear-attention / GDN-style bases do not expose the same KV-cache lever as dense-attention models.

Current Paper Intro: Story We Can Tell

Intro logic

Modern LLMs can accept longer contexts, but long-context use is not solved: full context is costly and sometimes harmful; simple reduction methods are brittle; and different tasks require different evidence-preservation strategies.

- Start with the field assumption: longer windows and context compression should help.
- Show the contradiction: the same method can win on retrieval and lose on semantic QA; full context can even hurt.
- Therefore the missing piece is not "one better compressor" but a mechanism that can **adapt to the evidence regime**.
- This motivates learned memory plus a reliability decision.

What v1.8.0 Already Gives Us

- A working learned-memory path on high-headroom tasks after fixing termination supervision and joint answer signal.
- A gate/fallback formulation that is useful conceptually: use memory when reliable, fall back when unsafe.
- Ablation evidence that the carrier is not fragile: many knobs move the compressed path by only a few points.
- A clearer negative result: the current gate story must become held-out or conformal, not in-sample threshold picking.

Takeaway

v1.8.0 is the prototype and evidence base. v2.0.0 should sharpen the claim into a cleaner reliability theory and a better necessity regime.

Next Paper Initialization Direction

Working title direction

Adaptive Evidence Memory for Long-Context Reasoning

- **Paper role of the fact-base:** establish that fixed black-box context reduction is not enough.
- **Paper role of the method:** introduce a memory that preserves evidence and a reliability signal that decides when to trust it.
- **Core novelty target:** self-verified memory + calibrated routing, not just another compressor.
- **Evaluation target:** public long-context regimes where window, RAG, prompt compression, and KV eviction fail for different reasons.

v2.0.0 Ideas

1. **Self-verified evidence memory:** train the memory to preserve evidence; use that preservation/reconstruction signal as the reliability check.
2. **Reliability-gated fallback:** use extracted knowledge when safe; otherwise fall back to raw context, retrieval, or a higher-cost path.
3. **Hybrid evidence memory:** combine a raw recent window with compressed global memory.
4. **Adaptive budget:** easy contexts get fewer memory tokens; dense or ambiguous contexts get more.

Next Experiments To Initialize

- **M1: Hybrid Evidence Memory** — reader sees raw recent window + compressed global memory.
- **M2: Conformal router** — held-out reliability calibration instead of in-sample threshold picking.
- **M3: Long-context necessity wave** — semantic-spread tasks plus retrieval honest-negative.
- **M4: Transfer matrix** — train on one domain, test on another; measure whether memory is domain-invariant.
- **M5: Figures** — turn the fact-base into length, task, and method-failure figures.

What Leaders Need to Decide

Option A: Fact-base paper Publish the long-context baseline map first; method comes later.

Option B: Method paper Build v2.0.0 now around self-verified evidence memory + conformal routing.

Option C: Two-stage plan Fact-base as Section 1 / motivation, then v2.0.0 as the method contribution.

Recommendation

Option C: use the fact-base to define the gap, then spend compute only on the minimum v2.0.0 experiments needed to show the gap can be closed.

Recommended Next Step

- Freeze the public long-context fact-base and produce the figures.
- Implement Hybrid Evidence Memory: raw window + compressed memory.
- Add conformal reliability routing and report risk–coverage / cost–coverage.
- Run the 5x5 transfer matrix to test cross-domain memory generalization.
- Decide whether v2.0.0 is a full method paper or a fact-base + position paper.

Ask

Approve GPU budget for the v2.0.0 public benchmark experiments, especially the hybrid-memory and transfer-matrix runs.

Reporting Period

Plan 08 / Long-Context Compression — July Week 1

Mon 2026-07-06 to Sun 2026-07-12

Focus: **Paper B (IMP)**; recall of Paper A. High-level logic here; details linked.

Details hub: OVERVIEW · fact-base · matrix-facts

Recall — the problem and the two papers

Problem (why it matters)

LLMs accept long inputs, but long context is not solved: full context is costly and sometimes *hurts*; training-free reduction methods are brittle; different tasks need different evidence.

Paper A compression **robustness via per-input validity-detection** — trust a compressed memory only when a self-verification signal says it's safe, else fall back to full (a "do-no-harm gate").

Paper B **observe long-context failures, then attach a lightweight importance-routing structure to a frozen base** (plug-and-play or lightly trained). *This week's focus.*

Full framing: OVERVIEW §2.

Preliminaries — terms (plain language)

- **Long-context / needle:** long input; the "needle" is the small fact that answers the question.
- **Frozen base:** pretrained LLM, weights unchanged — we add things around it, never retrain.
- **KV cache** / **KV-eviction**: attention's per-token memory / dropping cached tokens. **KV-free compression**: reduce the input itself (prune / merge / retrieve) — works on any architecture incl. cache-free linear models.
- **full / no_ctx:** references — whole input vs. no document (ceiling / floor).
- **Importance routing (IMP):** score token importance, keep the important, drop the rest, before the base reads.
- **query-relevance / surprisal:** how related to the question / how unexpected a token is.
- **span vs token:** keep whole contiguous spans vs. isolated tokens. **do-no-harm gate:** per-input trust-or-fallback.

Paper A — recall + a clean negative this week

Thesis

Robustness via a validity detector (gate): trust the cheap compressed memory only when safe; else fall back to full.

- **Reconstruction-as-gate rejected (honest):** a random-memory control shows the signal $\approx$ noise across a 15-cell ablation and a high-K bracket; the earlier "+6pt" was noise.
- **Root cause:** the learned soft-memory **can't compress extractive QA** — 4 recipes all give compress $\approx$ no_ctx ($\leq$0.20 vs full 0.69). Nothing to gate on.
- **Consequence:** ship the *confidence* gate; reconstruction blocked on a better compressor. Effort tilts to Paper B.

Detail: decisions D24–D26 · fact-base §12.3.

Paper B — observation → method

Observations driving the design

(1) No universal baseline; (2) naive merging kills the needle (ToMe = 0 on retrieval); (3) a cheap $O(L)$ signal finds the needle (query-relevance / surprisal); (4) attention-free is length-robust.

Method (IMP-v2.1.0, span-level, Mode A)

All results = `IMP-v2.1.0` (span-32, keep-0.5, signals query+surprisal); token-level v2.0 superseded.

Score each token by cheap signals ⇒ **keep the top-*p* important *spans* verbatim**, drop the rest ⇒ frozen base reads the short sequence. Modes: **plug-and-play** (no training) / **light-weight** (tiny distilled router + side-cache for linear models).

Full method + implementation: imp-method-and-implementation · spec: PAPER-B.

Paper B — results (high level)

- **Retrieval = full** at every length incl. 16k (where ToMe = 0, SnapKV collapses).
- **Token-level shreds coherent QA; span-level rescues it** (squad 0.15→0.46, hotpot 0.21→0.42) while keeping retrieval — the key ablation.
- **Beats full by denoising** on trivia and categorical; keep-rate curve is monotone.
- **Real-length LongBench (32k, no truncation):** on real long docs the base scores low and **compression often beats full** (necessity regime) — stronger than synthetic NIAH.

All numbers (main table, token-vs-span, keep curve, LongBench): fact-base §12–§12.3 · figures + narrative: OVERVIEW §3.1.

Analysis — who wins where, and IMP's honest standing

- **Retrieval:** kvzip / criticalkv / LLMLingua / RAG / IMP all $\approx$ full; ToMe/SnapKV/H2O fail.
- **Extractive/multi-hop QA: RAG is the strongest KV-free baseline**; window is a surprisingly strong trivial baseline when the answer is local.
- **IMP's honest position:** matches full on retrieval and recovers QA, but on *short-context* extractive QA it does **not yet beat RAG / LLMLingua**. Its distinctive value is **training-free · architecture-agnostic (incl. cache-free linear) · denoising · real-length wins** — not raw short-context accuracy.

Snapshot main table (all methods $\times$ benchmarks): fact-base §12.

Limitations & critical review (honest)

- No short-context accuracy win over RAG/LLMLingua yet; the case rests on generality + efficiency + long-context, which we must *show*.
- Prefill-saving is conditional (surprisal needs a full forward on quadratic bases unless a small scorer is used).
- **Not yet run on a linear/GDN base** — the one setting KV methods can't touch and IMP is uniquely useful.
- Equal-weight signal is sub-optimal; learned router (Mode B) unbuilt. narrativeqa/musr near base ceiling. Paper A blocked on the compressor.

Critical question

"Is IMP just a worse RAG?" — only defensible if we show (a) it works on cache-free linear models, (b) it wins in the necessity regime (input overruns the window), (c) semantic importance helps where lexical BM25-RAG fails. **These are the make-or-break experiments.**

Status & Next

Running a 12h / 6-GPU grand grid: full method $\times$ benchmark main table + all ablations (details land in the fact-base tomorrow).

Next plan (make-or-break first)

1. **Linear/GDN base** — IMP where KV methods don't exist (unique territory).
2. **Necessity regime** — inputs overrunning the window (LongBench / ∞Bench >100k).
3. **Semantic > lexical** — help on abstractive inputs where BM25-RAG fails.
4. Then: small draft-LM scorer (real prefill saving) · learned router (Mode B) · harvest grand grid $\rightarrow$ full table + figures.

Ask

Endorse the pivot to **Paper B (IMP)** as the primary line; keep Paper A's confidence-gate as secondary until a better compressor exists.

Artifacts & detailed data (all linked)

- OVERVIEW-both-papers-and-facts.md — narrative, figures, both papers, §3.1 new experiments.
- baseline-factbase-v2.0.0.md — every number: KV-free family (§10–11), IMP main table + ablations (§12), Paper-A compressor negative (§12.3).
- matrix-facts.md — one-line facts F1–F26 with evidence pointers.
- imp-method-and-implementation.md — IMP algorithm + exact code path.
- PAPER-B-v2.1.0-complete.md · PAPER-A-v1.8.1-complete.md — paper specs.
- decisions-2026-06-24.md — dated decision log (D1–D26).

Reporting Period

Plan 08 / Long-Context Compression — July Week 2

Mon 2026-07-13 to Sun 2026-07-19

This week: **Paper B rigor pass** (bug fix, corrected results, Occam v3.0) · **Paper C** (linear attention) opened · research deck.

First: a **full-project recall** (whole cycle, not just last week).

Recall (1/3) — background: the long-context memory wall

Why the project exists

LLMs accept long inputs, but long context is *not* solved: softmax attention keeps a **growing KV cache** (memory $\propto$ length), full context is costly and sometimes *hurts* accuracy, and training-free reduction methods are brittle and task-dependent.

- Two escape routes the field takes: **shrink the cache** (KV-eviction / prompt-compression / retrieval) vs. **remove the cache** (**linear attention** / SSM: a fixed-size recurrent state, $O(L)$ time, $O(1)$ memory).
- Our bases span both: **Qwen3** (quadratic) and **Qwen3.5/3.6** (Gated-DeltaNet linear 3:1 hybrid).

Guiding question (whole project)

On a *frozen* base, how do we feed / hold the *right* context so long-context accuracy survives — across architectures?

Recall (2/3) — one thesis, three papers (the whole cycle)

Thesis

Long-context accuracy is a **keep-the-answer-span** problem. No fixed compressor wins everywhere; the right mechanism is **input-adaptive** and ideally **training-free & architecture-agnostic**.

Paper A Learned **soft-memory** compressor on a frozen base + a do-no-harm gate. ⇒ **honest negative**: compressor is a commodity, gate a non-effect, can't compress extractive QA. *This motivated the pivot.*

Paper B Stop building a compressor; **diagnose** what works + attach a training-free **importance-routing prefilter (IMP)** to a frozen base. *Main result line.*

Paper C (v3.0) The diagnosis reproduces on **cache-free linear** models where KV-methods can't run ⇒ new line on **linear-attention forgetting / memory-state** (Qwen3.5/3.6 GDN family).

Deck: main-research · framing: OVERVIEW.

Recall (3/3) — what each paper established (to date)

Paper A Learned compressor ≈ no-context on extractive QA (4 recipes); reconstruction-gate ≈ chance; **training-free IMP beats the learned compressor**. ⇒ blocked; pivot justified. negatives

Paper B — diagnosis Full-test map over **9 model families**: (i) no universal compressor; (ii) "more context can hurt" (literary-MC full < blind); (iii) retrieval ≫ full on ultra-long; (iv) **model-invariant**, incl. linear GDN. facts F27/F31/F43

Paper B — method **IMP** = training-free, input-side, arch-agnostic router; keeps top spans/chunks verbatim; routes granularity per input. method

Paper C Landscape mapped (delta-rule family: DeltaNet→GDN→KDA→GDN-2); the linear **recall/forgetting bottleneck** is the opening. research line

This week (1/3) — Paper B: a rigor pass

- **Found & fixed a bug (F44).** IMP was *truncating* context to 16k *before* selecting, while RAG saw the whole 131k-token doc — 87% of RAG's kept content lived beyond 16k. Fix (full-doc retrieval): ∞Bench 51 $\rightarrow$ **76**, the -19 gap *reversed* to a win. audit
- **Re-ran the corrected main table** (ours = `auto`, full-N) + verified RAG reproduces. main table
- **Compared vs *all* baselines** (not just RAG): honest — KV-eviction wins literary-MC, full wins dense-reasoning; we own retrieval/extractive/needle/ultra-long *and* linear. best-of-all
- **Code review** (methods/benches/eval) + **Occam v3.0** (`IMP-lite`: one mode, one signal, one knob). review · v3.0

This week (2/3) — Paper C: linear-attention line opened

- **Research line + references** for linear attention & variants (Qwen3.5/3.6 GDN family only; no Qwen3 control). Landscape: delta-rule = "memory as online least-squares"; frontier = **how to erase vs write** the compressed state (GDN-2's channel-wise erase gate carries most of its gain). references · ablation design
- **GDN anti-forgetting modules (training-free, real benches): input-side compression rescues GDN on ultra-long** (∞Bench full 55 $\rightarrow$ imp-lite 76); recency-reorder modules are marginal/task-dependent. module results
- **Honesty:** the synthetic single-needle probe was inconclusive (task too easy / a probe bug we caught + reverted); trustworthy evidence is the real-bench grid. 128k probe OOMs (GDN kernel).

This week (3/3) — reporting deck + housekeeping

- **Research deck** (main-research): A/B/C overview, indexed, all artifacts GitHub-hyperlinked; total→detail structure with the mainline logic up front.
- **Scale/family directive (Paper C):** headline at **10–30B**, GDN family (Qwen3.5/3.6) only; secure the in-band **Qwen3.6-27B** checkpoint.
- **Cluster hygiene:** reclaimed a GPU held for 5 days by a stale demo server (mis-attributed as external); 6 clean GPUs now available.

Insights & next steps

Cross-cutting insights

Long-context = keep-the-gold-span (gold-recall predicts accuracy); no universal compressor; retrieval can *denoise above full*; the diagnosis is model-invariant; KV-eviction owns literary-MC but **can't run on linear** — that gap is ours.

Paper B run the IMP-lite Occam ablation; finalize v3.0; settle the ">RAG or RAG-configured" question.

Paper C secure Qwen3.6-27B + RULER (HF) access; diagnose GDN forgetting on real long-context; test input-side vs. erase-gate-retrofit fixes.